



72.20 - Redes de Información

TPE – CI/CD para el Despliegue y Gestión
de los Servicios (Helm & GitHub Actions)

Primer cuatrimestre 2026

Comisión S

Grupo 15

Integrantes:

LAUTARO JOAQUIN FARIAS – 60505

AXEL MARCELO CASTRO BENZA – 62358

ANDRES PODGORNÝ – 60570

Problemática y contexto

La aplicación a desplegar es "The Store", una plataforma de e-commerce conformada por una arquitectura de microservicios polígota (desarrollados en Java, Go y Node.js) diseñada para operar en un entorno de Kubernetes.

Problema: La gestión del despliegue en su estado actual presenta varias deficiencias para un entorno escalable:

- **Acoplamiento de manifiestos:** La configuración de todos los microservicios (`catalog`, `cart`, `checkout`, `orders`, `ui`) está concentrada en un único archivo denominado `dist/kubernetes.yaml`. Esto impide la actualización o modificación sencilla de parámetros individuales de cada servicio.
- **Falta de automatización y trazabilidad:** El ciclo de ejecución se delega a scripts imperativos locales (`local.sh`) que instancian el clúster usando Kind y ejecutan la compilación local y el despliegue con `kubectl apply`. Este esquema adolece de un pipeline automatizado, careciendo de trazabilidad clara del código a las imágenes en producción.
- **Imposibilidad de versionado ágil:** Este modelo obstaculiza el versionado semántico de los despliegues (releases) y aumenta severamente el riesgo ante el despliegue de cambios, haciendo un proceso de retorno seguro (rollback) riesgoso y manual.

Vale la pena destacar que el repositorio ya incluye un workflow de GitHub Actions (`.github/workflows/main.yml`) que crea un clúster Kind, aplica manifiestos y ejecuta pruebas E2E. La propuesta no parte de cero, sino que extiende ese enfoque.

Diseño de la solución

Se propone evolucionar a una arquitectura moderna y estándar de la industria, fuertemente orientada a las mejores prácticas de Integración Continua (CI) y Despliegue Continuo (CD). La solución se basa en dos pilares fundamentales: gestión de configuración (Helm) y automatización (GitHub Actions).

Gestión de Configuración: Helm

El manifiesto único y monolítico actual (`kubernetes.yaml`) será migrado hacia una estructura de Umbrella Chart en Helm.

- **Modularización:** Migrar desde el manifiesto único hacia un umbrella chart cuyos sub-charts correspondan a cada microservicio (`catalog`, `cart`, `checkout`, `orders`, `ui`) con sus templates propios (Deployments, Services, Ingresses, etc.).
- **Parametrización:** Al utilizar archivos `values.yaml` por servicio y por entorno, se inyectarán de manera dinámica valores clave que hoy son rígidos. Esto incluye configuración de manejo de réplicas, control de recursos (CPU/RAM limits & requests), variables de entorno, y puntos de montaje de volúmenes.

- **Ciclo de Vida:** Uso de instalaciones y actualizaciones como releases de Helm (`helm install/helm upgrade`), con historial usable para auditoría y rollback.

Automatización: GitHub Actions

Se centralizará la orquestación en GitHub Actions a través de un workflow que detectará cambios en las ramas principales para ejecutar dos fases críticas:

1. Integración Continua (CI):

- **Validación Estática:** Lint tanto de los Dockerfiles como de los templates de Helm en push y en pull request.
- **Construcción y Versionado:** El pipeline compilará en paralelo el código usando builds multi-etapa de Docker. A los artefactos resultantes se les asignará un tag semántico automático.
- **Registro:** Se publicarán estas imágenes inmutables en el GitHub Container Registry (GHCR) como artefactos seguros para el siguiente paso.

2. Despliegue Continuo (CD):

- **Despliegue hacia clúster local:** el job de CD se ejecuta en un self-hosted runner (agente de GitHub Actions) con acceso al kubeconfig del clúster Kind local. Ese job ejecuta `helm install/upgrade` apuntando a imágenes publicadas en GHCR (p. ej. por tag de commit) y, de esta forma, conecta el pipeline con el clúster sin requerir que el orquestador acceda a la red privada.
- **Estrategias de Actualización:** Los Deployments usarán `RollingUpdate`. Kubernetes sustituye pods gradualmente según `readiness`, lo que permite despliegues sin ventana única de mantenimiento siempre que los recursos lo permitan.

POC y casos de uso

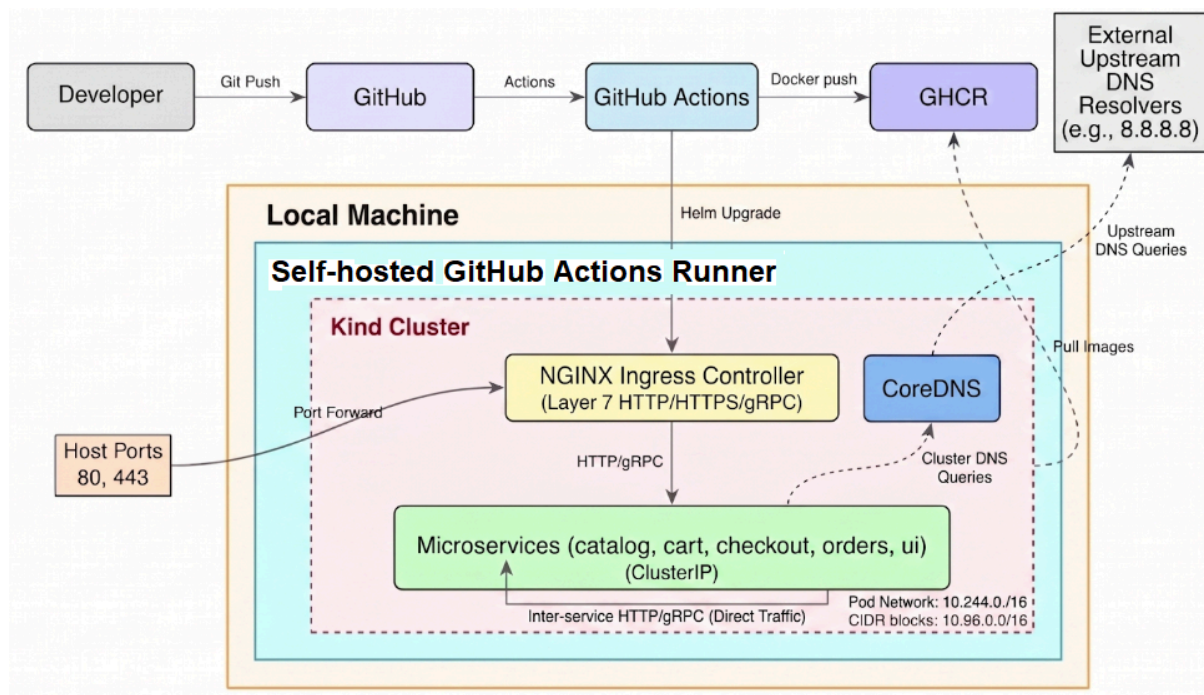
Este Proof of Concept (POC) busca establecer una validación del flujo automatizado diseñado, delimitando el trabajo sobre tres escenarios principales:

- **Caso 1: Cambio acotado a un microservicio.** Se introduce un cambio exclusivo en el servicio `catalog` (código o solo nueva imagen). Se verificará que el pipeline priorice el build y publicación del artefacto afectado y que el `helm upgrade` solo modifique los recursos cuyo manifiesto cambió. Los demás servicios no deberían ser afectados si sus especificaciones no cambian.
- **Caso 2: E2E como compuerta.** A fin de evitar fallas o builds inestables, el pipeline instrumentará la ejecución íntegra de la suite de pruebas mediante el script provisto (`local.sh e2e-test`) luego del despliegue en el clúster transitorio de CI. Como compuerta de protección, si la pasarela de prueba E2E llegara a fallar, el pipeline concluirá reportando error y bloqueará cualquier tipo de promoción o consideración de dicho release como estable.

- **Caso 3: Gestión ante desastres (Rollback).** Se simula un despliegue defectuoso (por ejemplo imagen errónea o sondas que impiden el readiness). Se demostrará el uso del comando `helm rollback` para volver a la revisión anterior registrada por Helm, apoyándose en el historial de releases y en imágenes inmutables en GHCR.

Diagrama de arquitectura

El modelo refleja la infraestructura general, CI/CD y los subsistemas de la red virtual del orquestador:



- **CI/CD:**
 - **Runners:** Entorno Ubuntu efímero provisto de forma automatizada por GitHub Actions, encargado de orquestar el flujo pero actuando desde afuera del nodo cliente.
 - **GHCR:** Almacenaje inmutable de las capas Docker por servicio en GitHub Container Registry (GHCR).
 - **Cluster de Operación (En Local Machine):** Servidor de Kubernetes apoyado en Kind (Kubernetes IN Docker) que reside de manera persistente o externa sobre una máquina local. El runner de GitHub Actions no lo regenera en sí, sino que se conecta y ejecuta el comando `helm upgrade` para impactarlos sin mutar su ubicación subyacente.
- **Flujo de Despliegue:**
 - Ante la actualización del manifiesto a través del pipeline, el propio Cluster Local es el responsable de ejecutar un pull de las nuevas imágenes desde GHCR hacia el host, tal como lo indica la relación punteada, antes de iniciar la rotación de pods.

- **Diseño de Red y Direccionamiento Lógico (Networking Local):**
 - **Pod Network (Bloque CIDR):** 10.244.0.0/16. Subred por defecto que emplea el plugin CNI de Kind dentro del entorno local subyacente para repartir las direcciones IP a los contenedores y asegurar comunicación.
 - **Service Network (Bloque CIDR):** 10.96.0.0/16. Subred abstraída a Kubernetes de la Máquina Local (ClusterIPs).
- **Protocolos y Exposición (Ingress & Egress):**
 - **Ingress:** NGINX Ingress Controller, en Kind, el script local mapea 80/TCP y 443/TCP del host al control plane gestionando tráfico HTTP/HTTPS (capa 7).
 - **Pod-to-Pod Traffic:** La comunicación interservicios emplea resolución DNS a través de CoreDNS, resolviendo peticiones en HTTP plano y gRPC entre las IPs de nivel pod (10.244.x.x).

Alternativas consideradas

- **Kustomize vs Helm:** Se evaluó Kustomize, dada su inclusión natural de la mano de `kubect1` ("template-free"). Fue finalmente descartado ya que Helm ofrece paquetizado, valores por entorno y una gestión muy prolija del ciclo de vida de los "releases" de aplicaciones modulares. Todo esto nos facilitará la prueba de concepto del caso 3 mencionado anteriormente.
- **Jenkins / GitLab CI vs GitHub Actions:** Desplazar la solución CI/CD a herramientas como Jenkins requeriría un mantenimiento y operación manual de instancias/servidores en red. Por el contrario, GitHub Actions se escogió para disminuir cualquier barrera en cuanto a infraestructura de automatización (runners autogestionados) integrándose nativamente y sin costes agregados en el repositorio y los registros GCHR.